Hyperparameters in Decision Trees and How to Use Them

Abstract

Decision Trees are among the most widely used machine learning algorithms because of their simplicity and interpretability. However, their performance depends heavily on the choice of hyperparameters that control how the tree grows and splits the data. If these parameters are not tuned properly, the model may suffer from overfitting or underfitting. This research discusses the main hyperparameters used in Decision Tree models, explains their role in controlling model complexity, and provides guidance on when and how to use each parameter.

---

1. Introduction

In **Machine Learning**, model performance is influenced by two types of parameters:

- **Model Parameters** – learned from the training data
- **Hyperparameters** – set before training and control the learning process

In **Decision Tree**, hyperparameters determine:

- the depth of the tree
- the number of samples needed to split nodes
- how splits are selected

Proper tuning of these hyperparameters helps balance the trade-off between **bias and variance**.

---

2. Important Hyperparameters in Decision Trees

2.1 max_depth

Definition

`max_depth` controls the **maximum depth of the decision tree**.

Effect

- Large depth → complex tree → risk of **overfitting**
- Small depth → simpler tree → risk of **underfitting**

Example

```
DecisionTreeClassifier(max_depth=5)
```

When to Use

Use **smaller values** when:

- dataset is small
- model overfits training data

Use **larger values** when:

- dataset is large
- relationships between features are complex

2.2 min_samples_split

Definition

`min_samples_split` determines the **minimum number of samples required to split a node**.

Example

```
DecisionTreeClassifier(min_samples_split=10)
```

Effect

- Small value → tree splits frequently → overfitting
- Large value → fewer splits → simpler model

When to Use

Increase this value when:

- tree grows too large
- model memorizes training data

2.3 min_samples_leaf

Definition

`min_samples_leaf` specifies the **minimum number of samples that must exist in a leaf node**.

Example

```
DecisionTreeClassifier(min_samples_leaf=5)
```

Effect

Higher values:

- smoother predictions
- reduce variance

Lower values:

- more flexible model
- may overfit

When to Use

Use higher values when:

- dataset contains noise
- model is unstable

2.4 max_features

Definition

`max_features` controls the **number of features considered when splitting a node**.

Example

```
DecisionTreeClassifier(max_features="sqrt")
```

Possible values:

Value	Meaning
None	use all features

Value	Meaning
sqrt	√number of features
log2	log2(number of features)

Effect

Lower values:

- increase randomness
- reduce overfitting

Higher values:

- increase model accuracy
- risk overfitting

2.5 criterion

Definition

`criterion` determines **how the quality of a split is measured**.

Common criteria:

Criterion	Description
gini	Gini impurity
entropy	information gain

Example:

```
DecisionTreeClassifier(criterion="gini")
```

Gini Impurity

Used to measure how mixed the classes are.

Formula:

Gini=1−∑pi2Gini = 1 - \sum p_i^2Gini=1−∑pi2

Where:

pip_ipi = probability of class i.

Entropy

Based on information theory.

Formula:

Entropy=−∑pilog2(pi)Entropy = -\sum p_i \log_2(p_i)Entropy=−∑pilog2(pi)

When to Use

Criterion	When to Use
Gini	faster and commonly used
Entropy	when information gain interpretation is needed

2.6 max_leaf_nodes

Definition

Limits the **maximum number of leaf nodes**.

Example:

```
DecisionTreeClassifier(max_leaf_nodes=20)
```

Effect

- controls tree complexity
- prevents large trees

2.7 min_impurity_decrease

Definition

Minimum impurity decrease required for a split.

Example:

```
DecisionTreeClassifier(min_impurity_decrease=0.01)
```

Effect

- prevents small useless splits
- improves generalization

---

3. Hyperparameter Tuning

Hyperparameters are usually optimized using techniques such as:

- Grid Search
- Random Search
- Cross-validation

Example using **Grid Search**:

```
from sklearn.model_selection import GridSearchCV

params = {
    "max_depth":[3,5,7,10],
    "min_samples_split":[2,5,10],
    "min_samples_leaf":[1,2,4]
}

grid = GridSearchCV(DecisionTreeClassifier(), params, cv=5)
grid.fit(X_train, y_train)
```

---

4. Overfitting vs Underfitting

Decision trees easily overfit when they grow too deep.

Solutions:

- limit `max_depth`
- increase `min_samples_leaf`
- increase `min_samples_split`

---

5. Practical Example

Example model:

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    max_depth=5,
    min_samples_split=10,
    min_samples_leaf=3,
    criterion="gini"
)

model.fit(X_train, y_train)
```

This configuration produces a balanced tree with controlled complexity.

---

6. Conclusion

Hyperparameters play a critical role in controlling the behavior of decision tree models. Proper tuning helps prevent overfitting and improves model generalization. Parameters such as `max_depth`, `min_samples_split`, and `min_samples_leaf` control tree structure, while `criterion` determines how splits are evaluated. Selecting appropriate hyperparameters through systematic tuning methods leads to more accurate and reliable machine learning models.

References

1. https://scikit-learn.org/stable/modules/tree.html
2. https://machinelearningmastery.com
3. https://towardsdatascience.com
4. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow
5. Scikit-learn documentation